

DAT 1기 캡스톤 프로젝트 보고서

선형회귀 분석과 다양한 머신러닝 기법을 활용한 배달 소요시간 예측 프로젝트

:선형회귀, 릿지, 라쏘, 엘라스틱넷, 앙상블 기법간

모델 예측 성능 비교

DAT 1기

3각김밥

김가영, 김선민, 김예진



Data Analysis & Techonlogy

DAT 1기 캡스톤 프로젝트 보고서

선형회귀 분석과 다양한 머신러닝 기법을 활용한 음식배달 소요시간 예측 프로젝트

:선형회귀, 릿지, 라쏘, 엘라스틱넷, 앙상블 기법간
모델 예측 성능 비교

FOOD Delivery Time prediction project, Using linear conversion analysis
and various machine learning techniques

이 보고서를 캡스톤 프로젝트 보고서로 제출합니다.

2023년 06월 08일

DAT 1기

3각김밥

김가영, 김선민, 김예진



Data Analysis & Techonlogy

DAT 1기 캡스톤 프로젝트 보고서

선형회귀 분석과 다양한 머신러닝 기법을 활용한

배달 소요시간 예측 프로젝트

: 선형회귀, 릿지, 라쏘, 엘라스틱넷, 앙상블 기법간

모델 예측 성능 비교

국문 요약

배달어플을 사용한 음식 배달 서비스의 사용은 꾸준히 늘고 있는 가운데, 배달앱 입점 음식점들이 안내하는 배달 예상 시간이 실제 주문 후에 더 늘어나는 경우가 비일비재해 소비자들이 불만을 토로하고 있습니다. 이에 배달 소요시간에 영향을 미치는 변수들에 대해 알아보고, 음식 배달 소요시간을 예측하고자 합니다. 선형 회귀와 선형회귀의 패널티를 더한 릿지, 라쏘, 엘라스틱넷 그리고 앙상블 기법인 랜덤 포레스트까지 총 5개의 모델을 사용하였으며 최상의 예측 모델을 찾고자 하였습니다. 합리적인 변수 선출 과정을 통해 변수를 선정하였으며 최적의 하이퍼파라미터를 찾는 데에 집중하였습니다. 그 결과, 큰 성능 차이는 없었지만 앙상블 모델인 랜덤포레스트의 성능이 가장 좋았음을 알 수 있었습니다.

Abstract

While the use of food delivery services using delivery apps is steadily increasing, consumers are complaining that the estimated delivery time guided by restaurants in delivery apps often increases after actual orders. Therefore, we would like to find out the variables that affect the delivery time using the dataset collected from the CGL site and predict the delivery time. We used a total of five models, including Ridge, Lasso, Elasticnet, and Random Forest, an ensemble technique with linear regression and linear regression penalties, and we wanted to find the best predictive model. We selected variables through a reasonable variable selection process and focused on finding the optimal hyperparameters. As a result, there was no significant difference in performance, but the ensemble model Random Forest showed the best performance.

DAT 1기

3각김밥

김가영, 김선민, 김예진



Data Analysis & Techonlogy

목 차

제 1장 서론	3
제 1절 연구 배경	3
제 2절 연구 목적 및 보고서 구성	3
제 2장 데이터 소개	4
제 1절 조사 및 분석 데이터 소개	4
제 2절 데이터 전처리	6
제 3장 분석 결과	8
제 1절 연구 모형 설계	8
제 2절 모델 적용	12
제 4장 결론	15
제 1절 연구 결과 종합 및 시사점 도출	15
제 2절 한계점 및 향후 연구 방향성 제언	16
참고문헌	16

표 목차

[표 1-1]	5
[표 2-1]	6
[표 3-1]	8
[표 3-2]	13
[표 3-3]	15

그림 목차

[그림 3-1]	9
[그림 3-2]	10
[그림 3-3]	13
[그림 3-4]	14
[그림 4-1]	16

제 1장 서론

제 1 절 연구 배경

4 차 산업혁명 시대와 코로나 19 의 영향으로 비대면 라이프 스타일이 가속화됨에 따라 배달 플랫폼을 기반으로한 배달 외식업이 급성장하였다. 외식을 하기보다는 스마트폰의 배달 플랫폼 애플리케이션(이하 배달 앱)을 통하여 음식점, 카페, 디저트의 리뷰와 사진 등 정보를 검색하여 배달 음식을 집에서 먹는 일이 많아졌다. 배달 앱에서는 이용가능한 지역을 설정하면 그 지역에서 이용 가능한 음식점들을 보여주며, 음식점마다 '최소 주문금액', '결제방법', '배달시간' 등을 주요 정보로 우선 제공하고 있다. 하지만, 앱에 표시되는 '배달 예상 시간'이 주문 후 실제 배달 소요 시간과 차이가 발생하면서 소비자 불만이 발생한다.

배달앱들은 소비자가 주문하면 업체까지의 거리와 주문을 받은 업체가 설정한 조리시간을 책정해 배달 소요시간을 안내하고 있다. 하지만, 거리 이외에도 다른 요인들이 배달 소요시간에 영향을 미칠 수 있다. 본 연구에서 사용되는 '교통혼잡도', '동시 배달 수' 등으로 인한 주문 지연 등이 예상치 못한 배달 지연의 원인이 될 수 있다.

또한, 배달 앱에서 소비자에게 제공하는 소요시간보다 실제 배달 시간이 적게 소요될 경우, “안내드린 시간보다 30분 빨리 배달되었습니다.”와 같은 메시지가 전달된다. 소비자들에게 이러한 메시지는 해당 업체에 대한 긍정적인 이미지를 가지게끔 하며, 이는 업체의 평판에 직접적으로 영향을 미치는 리뷰와도 관련이 지어진다. 그로 인해, 실제 배달 시간보다 일부러 과대측정하여 배달 예상 소요시간을 선택하는 경우가 발생할 수 있다. 따라서, 업체가 설정한 조리시간과 거리 만을 기준으로 배달 소요시간을 정확히 예측하는 현재의 배달 소요시간 예측은 객관성 있다고 보기 어렵다.

제 2 절 연구 목적 및 보고서 구성

우리는 이 연구를 통해서 배달 소요 시간을 배달 기사 정보, 위치정보, 배달 시간 및 계절 정보, 배달 수단과 음식의 유형, 배달 시킨 곳의 축제 여부, 교통 혼잡도, 배달 픽업 시간 변수에 따라 예측해보려고 한다. 이 예측 결과를 통해, 판매자는 예상 도착시간을 고려하여 주문을 조절하고, 조리 및 배달 절차를 최적화 할 수 있으며, 고객 만족도를 향상시킬 수 있다.

실제 도착 시간과 예측이 일치하면 주문자는 신뢰를 가지고 서비스를 이용할 수 있게 되며, 이는 곧 업체의 리뷰, 평판, 나아가 매출과 연관될 수 있다. 또한, 판매자가 주문 처리와 배달 업무를 이행하는 데에도 도움이 된다. 예상 도착시간을 고려하여 주문을 조절하고, 조리 및 배달 절차를 최적화 할 수 있으며, 이는 업무 효율성과 비용 절감에도 기여할 수 있다. 주문자의 입장에서 배달 소요시간을 예측함으로써 예상보다 오랜 시간을 기다려야 하는 경우를 사전에 인지 할 수 있다.

본 연구의 구성은 다음과 같다. 2 장에서는 분석에 사용된 데이터 변수에 대한 설명으로, 자료수집 및 데이터 전처리 과정에 대한 설명을 한 후 변수 시각화를 통한 분석이 진행된다. 3 장에서는 예측 모형에 대한 설명으로, 파이썬을 이용하여 진행한 머신러닝 결과를 보여준다. 예측에 사용한 모델로는 선형회귀, 릿지, 라쏘, 엘라스틱넷, 앙상블 기법(랜덤포레스트) 총 다섯 가지 모형이다. 각각의 모델링 결과를 잔차를 통해 살펴보고 최적의 모델을 찾기 위한 변수 조정과정을 설명한다. 4 장은 연구 결론부분으로, 모델 평가 지표를 통해 선정한 최종 예측 모델을 가지고 배달 소요시간을 예측하여 실제 값과 비교한다. 이 연구가 가지는 시사점을 살펴보고, 추후 과제에 대해 짚어보며 보고서를 마무리한다.

제 2장 데이터 소개

제 1절 조사 및 분석 데이터 소개

1.1 자료수집

분석에 사용된 자료는 데이터 과학 및 머신러닝 경진대회를 주최하는 온라인 커뮤니티인 Kaggle 사이트(<https://www.kaggle.com>)에서 제공하는 오픈 데이터인 '배달 음식 데이터셋 (Food Delivery Dataset)'을 사용하였다.

1.2 변수 소개

해당 데이터는 음식 배달 서비스를 이용했을 때 수집되는 정보들에 관한 데이터로, 배

달 기사 정보, 식당과 배달시킨 장소의 위치정보, 배달시킨 날짜의 시간 및 계절 정보, 배달 수단과 음식의 유형 등의 변수들이 제공된다. 이외에도 배달 시킨 곳의 축제 여부, 교통 혼잡도, 배달 픽업 시간의 변수가 담겨 있으며, 예측하게 될 타겟 변수인 배달 소요 시간 변수까지 총 20개의 변수들로 구성되어 있다.

총 관측치의 개수는 45593개였으며, 변수들의 유형은 숫자형, 범주형 변수로 구성 되어 있었다. 하지만, 변수 별로 타입이 맞지 않았고 결측치가 존재하였기에 데이터를 전처리 하는 작업이 수행되어야만 했다.

[표 1-1] 전처리 이전의 원본 데이터셋

X(독립변수)	변수설명	유형
ID	배달 시킨 사람 ID	object
Delivery_person_ID	배달 기사 ID	object
Delivery_person_Age	배달 기사 나이	object
Delivery_person_Ratings	배달 기사의 평점	object
Restaurant_latitude	식당 위도	float64
Restaurant_longitude	식당 경도	float64
Delivery_location_latitude	배달 장소 위도	float64
Delivery_location_longitude	배달 장소 경도	float64
Order_Date	배달 시킨 날짜	object
Time_Orderd	배달 시킨 시간	object
Time_Order_picked	배달 음식을 챙긴 시간	object
Weatherconditions	날씨 상태('Sunny', 'Stormy', 'Sandstorms', 'Cloudy', 'Fog', 'Windy')	object
Road_traffic_density	교통 혼잡도 ('High ', 'Jam ', 'Low ', 'Medium ')	object
Vehicle_condition	배달 수단 상태(2, 0, 1, 3)	int64
Type_of_order	배달 시킨 음식의 종류('Snack ', 'Drinks ', 'Buffet ', 'Meal ')	object
Type_of_vehicle	배달 수단('motorcycle ', 'scooter ', 'electric_scooter ', 'bicycle ')	object
multiple_deliveries	동시 배달 수 ('0', '1', '3', '2')	object
Festival	축제 ('No ', 'Yes ')	object
City	도시 여부 ('Urban ', 'Metropolitan ',	object

	'Semi-Urban ')	
Y(종속변수)	변수설명	타입
Time_taken(min)	배달 소요 시간(분)	object

제 2절 데이터 전처리

2.1 수치형 데이터화

[표 1-1]은 원본 데이터의 변수형을 도출한 결과로, Delivery_person_Age를 포함한 수치형 변수들의 유형이 범주형으로 분류되어 있었음을 확인할 수 있다. 따라서, 범주형 데이터들을 숫자 데이터로 바꿔주는 작업을 진행하였다.

2.2 결측치 처리 및 대체

[표 2-1]은 결측치가 나타난 변수와 해당 결측치 제거 방법을 나타낸 표이다. 결측치의 경우, pandas 패키지의 isnull() 함수를 통해 확인해보니 결측치가 발견되지 않았지만 데이터셋에는 'NaN' 으로 결측치가 존재함을 확인하였다. 따라서, 'NaN'을 결측치로 대체 시킨 후 확인해보니, 'Delivery_person_Age', 'Delivery_person_Ratings', 'Time_Orderd', 'Weatherconditions', 'Road_traffic_density', 'multiple_deliveries', 'Festival', 'City'에서 결측치가 발견되었다. 결측치가 제일 많았던 'Delivery_person_Ratings'와 'Delivery_person_Age'는 평균치로 대체 하였고, 나머지 'Time_Orderd', 'City', 'multiple_deliveries', 'Road_traffic_density', 'Weatherconditions', 'Festival'의 결측치들은 제거하였다. 이로써 모든 결측치를 제거하는 과정을 거쳤다.

[표 2-1] 결측치 처리 방식

결측치가 존재하는 변수	결측치 개수	결측치 비율	결측치 제거 방법
Delivery_person_Ratings	1,908	0.041849	평균치로 대체
Delivery_person_Age	1,854	0.040664	평균치로 대체
Time_Orderd	1,731	0.037966	제거
City	1200	0.026320	제거
multiple_deliveries	993	0.021780	제거
Weatherconditions	616	0.013511	제거

Road_traffic_density	601	0.013182	제거
Festival	228	0.005001	제거

2.3 새로운 변수 생성

배달 소요 시간은 시간대에 굉장히 많은 영향을 받는다. 일반적으로 식사시간대인 점심 시간대(12시~1시)와 저녁시간대(17시~20시)에 주문이 몰리게 되면 배달 소요시간이 많아지게 되고, 다른 요일에 비해 금요일에 배달 주문이 많아질 수도 있으며 월드컵이나 올림픽 같은 경기나 축제가 있는 달에 따라 배달 주문량이 차이가 날 수도 있다고 생각하였다.

따라서 배달 시킨 날짜를 의미하는 'Order_Date' 변수를 가지고 사용하여 연도별, 월별, 일별 데이터로 나눠 새로운 컬럼을 생성하였다. 'Order_Date'는 Year-Month-day 형태로 저장되어 있었기에, datetime 모듈을 활용하여 Order_Year, Order_Month, Order_Day 컬럼을 생성하였고 요일을 뜻하는 Order_week 변수까지 생성 할 수 있었다. 하지만 Order_Year의 경우, 전부 2022년 데이터로 모든 행의 값이 같았기에 추후에 제거하였다.

또한 소비자가 배달 음식을 주문한 시간과 배달원이 배달 음식을 챙긴 시간과 차이를 계산하여 'diff_picked' 라는 이름의 변수로 생성하여 소비자가 주문한 시간으로부터 배달원이 배달 음식을 챙기기까지의 과정이 얼마나 소요되었는지 고려하였다.

2.4 범주형 변수의 더미변수화

범주형 데이터의 경우, 모델에 적용하기 위해서는 수치형 데이터로 변환하는 것이 필요하다. 따라서 get_dummies() 함수를 이용하여 범주형 변수였던 'Weatherconditions', 'Road_traffic_density', 'Type_of_order', 'Type_of_vehicle', 'Festival', 'City', 'order_week'를 수치형 데이터로 변환해주었다.

2.5 변수 삭제

'ID', 'Delivery_person_ID'와 같이 이번 프로젝트에서 쓰이지 않을 것이라 판단된 변수들과 위치정보들은 추후에 사용하게 될 가능성이 있기에 새로운 변수에 저장한 뒤 훈련데이터셋에서는 제거하였다. 또한 새로운 변수들을 생성하는데 쓰인 변수들과 앞서 더미변수를 만들면서 생성된 컬럼들 중 1개를 삭제하였다. 이로써 1차 전처리 작업을 거쳐 총 30개의 변수와

41,616개의 관측치로 재구성 하였다.

제 3장 분석 결과

제 1절 연구 모형 설계

이번 장에서는 변수 선출 방식을 통해 변수들을 선정한 후, 다중선형회귀분석, 릿지, 라쏘, 엘라스틱넷 그리고 앙상블 기법인 랜덤포레스트 기법을 이용하여 관측치 별 배달 소요 시간을 예측해보도록 한다. 예측성능을 비교하기 위해, 41,616 개의 자료를 8:2 비율로 train과 test로 나누어 각각 train 33292 개, test 8,324 개 의 자료로 분석을 진행하였다. 평가지표로는 평균 제곱 오차인 RMSE를 통해 모델 성능을 각 모델들간의 성능을 비교하고 이를 해석하고자 한다.

1.1 정규화, 표준화 진행 및 다중공선성 확인

전처리를 거친 데이터셋을 OLS와 LinearRegression에 먼저 적용시켜보았다. 정규화 & 표준화 이전의 R^2 는 0.941, RMSE는 6.07654 라는 결과가 나왔고, 수치형 변수들의 정규화와 표준화를 거친 후 다시 모델에 적용시켜본 결과 R^2 는 0.822, RMSE는 0.12517 이었다.

[표 3-1] 정규화 및 표준화 진행 여부에 따른 모델 성능 비교표

	adjusted- R^2 (OLS)	RMSE (LinearRegression)
정규화 & 표준화 이전	0.941	6.07654
정규화 & 표준화 이후	0.822	0.151743

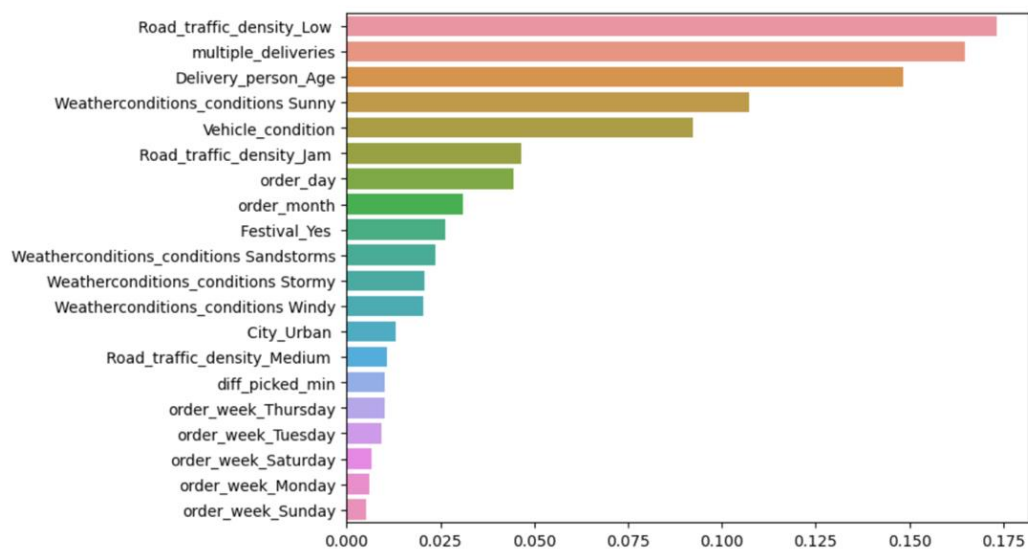
정규화와 표준화를 거친 데이터의 다중공선성이 존재하는지 확인하기 위해 VIF지표를 이용하였다. 그 결과, 가장 높은 수치를 보인 변수는 26.021287의 Delivery_person_Ratings, 그 다음으로는 Type_of_vehicle_motorcycle이 9.666, Type_of_vehicle_scooter이 4.73였다. Delivery_person_Ratings는 일반적으로 높다고 판단되는 기준인 10을 넘는 수치이기에 다른 변수까지 설명하고 있다고 판단되어 이를 제거하였다. 다중공선성이 가장 높은 변수

'Road_traffic_density_Jam', 'Road_traffic_density_Low', 'City_Urban', 'City_Semi-Urban', 'Weatherconditions_conditions_Sunny', 'Weatherconditions_conditions_Sandstormy', 'Weatherconditions_conditions_Stormy', 'Weatherconditions_conditions_Windy'으로 총 12개의 독립변수를 사용하기로 하였다

1.3.2. 랜덤포레스트 변수 중요도

랜덤포레스트를 이용하면 가장 예측력이 강한 변수들을 확인할 수 있기에 이번 프로젝트에서 변수를 선정하는데에 사용하였다. 그리고 변수 20개를 중요도 순서로 시각화 한 결과는 다음 [그림 3-2]와 같았다. 설정한 하이퍼파라미터는 $n_estimators=40$, $max_depth=15$ 로 하였다.

[그림 3-2] 랜덤포레스트의 변수 중요도 (significant level = 0.05)



1.3.3. 최종 변수

Stepwise selection을 통해 선정된 12가지 변수와 랜덤포레스트의 변수 중요도를 통해 선정된 20가지의 변수 집단간 교집단인 'multiple_deliveries', 'Delivery_person_Age', 'Road_traffic_density_Jam', 'Vehicle_condition', 'Weatherconditions_conditions_Stormy', 'Weatherconditions_conditions_Sunny', 'Festival_Yes', 'Road_traffic_density_Low', 'City_Urban',

'Weatherconditions_conditions_Sandstormy', 'Weatherconditions_conditions_Windy'을 최종 설명 변수로 하였다. 따라서 최종 데이터 셋은 41616 개의 자료와 11가지의 변수들로 이루어졌다.

제 2절 모델 적용

이번 2절에서는 앞서 두 번의 변수선출 과정으로 인해 최종적으로 구성된 데이터셋을 가지고 5개의 모델에 각각 적용해본 뒤, 모델의 정확도와 RMSE를 평가지표로 하여 비교해보고자 한다.

2.1 다중 선형회귀

선형 회귀(Linear Regression)는 회귀 문제를 예측할 때 사용하는 알고리즘 중 하나이다. '독립 변수가 커질 때 종속 변수가 크거나 작게 변하는' 관계를 모델링 하는 것이다. 선형회귀는 보통 하나 이상의 독립 변수를 사용하여 모델링 하는데 이 때, 독립 변수가 둘 이상이면 '다중선형회귀(multiple linear regression)'라고 수식은 아래와 같다.

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_k x_{ik} + \epsilon_i$$

다중선형회귀 모형에 적용해보고 회귀계수를 분석한 결과, 배달 기사님의 나이가 많을수록 동시에 배달하는 음식의 개수가 많을수록, 축제 중일 수록 배달 소요시간이 증가하는 양의 상관관계를 보였다. 반면에, 날씨가 맑을수록, 도심 속 일수록, 교통 밀도가 낮을수록 배달 소요시간이 감소하는 음의 상관 관계를 보였다. 선형회귀 모델의 경우 정확도는 0.49026였으며 RMSE는 0.151743가 도출되었다.

2.2 릿지, 라쏘, 엘라스틱넷

선형 모델의 비용 함수는 실제 값과 예측 값의 차이를 최소화하는 것만 고려하였다. 그러다보니 과적합이 발생하여 회귀 계수가 쉽게 커지게 된다. 따라서, 과적합을 방지하기 위해 회귀 계수가 커지지 않도록 비용함수를 최소화시켜야 한다.

비용 함수를 최소화 하는 대표적인 규제 선형 모델로는 계수의 제곱을 최소화 시키는

L2 규제 방법인 릿지(Ridge) 회귀 방법과 계수의 절대값을 줄이는 L1 규제 방법인 라쏘(Lasso) 회귀 방법 그리고 이 두가지 규제방법을 혼합한 엘라스틱넷(ElasticNet)을 사용한다. 아래는 릿지와 라쏘 그리고 엘라스틱넷의 비용함수에 관한 수식이다.

$$\sum_{i=1}^M (y_i - \hat{y}_i)^2 = \sum_{i=1}^M \left(y_i - \sum_{j=0}^p w_j \times w_{ij} \right)^2 + \lambda \sum_{j=0}^p w_j^0$$

$$\sum_{i=1}^M (y_i - \hat{y}_i)^2 = \sum_{i=1}^M \left(y_i - \sum_{j=0}^p w_j \times w_{ij} \right)^2 + \lambda \sum_{j=0}^p |w_j|$$

$$\frac{\sum_{i=1}^n (y_i - x_i^J \hat{\beta})^2}{2n} + \lambda \left(\frac{1-\alpha}{2} \sum_{j=1}^m \hat{\beta}_j^2 + \alpha \sum_{j=1}^m |\hat{\beta}_j| \right)$$

2.1.1 모델 성능 비교

세 모형에서 알파값(α)이 각각 0.001, 0.005, 0.01, 0.05, 0.1, 1, 10 일 때의 각 모델 정확도와 RMSE를 비교해보았다. 릿지 모델의 경우, 알파값에 따라 모델성능 부분에서 큰 차이가 없었으나, 중요하지 않은 변수의 회귀계수를 0으로 보내버리는 Lasso와 Elasticnet에서는 α 이 각각 0.05와 0.1이 넘어가자 정확도가 떨어지고 RMSE 또한 커지는 양상을 보였다.

이는 규제항의 모수인 값이 높아지면 추정 계수가 0에 가까워지고 모형은 단순화 됨을 의미한다. 세 모델 모두 α 은 가장 작을 때인 0.001을 사용하였기에 이는 표준 선형회귀모형과 거의 같은 형태를 띄게된다.

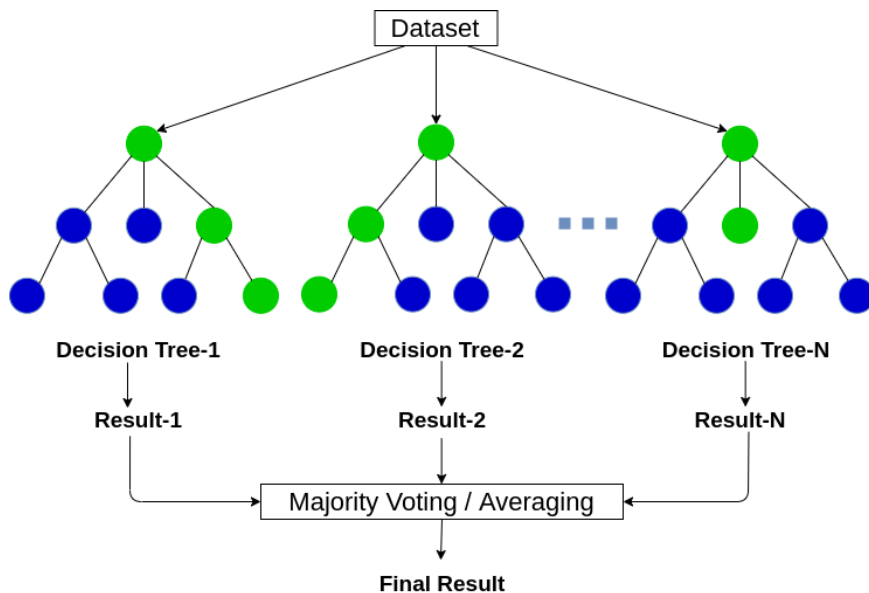
[표 3-2] 알파값(α) 별 Lidge, Lasso, Elasticnet의 모델 정확도 및 RMSE 비교표

		0.001	0.005	0.01	0.05	0.1	1	10
Lidge	정확도	0.49	0.49	0.49	0.49	0.49	0.49	0.49
	RMSE	0.1517	0.1517	0.1517	0.1517	0.1517	0.1517	0.1517
Lasso	정확도	0.487	0.429	0.335	0.0	0.0	0.0	0.0
	RMSE	0.1521	0.1605	0.1733	0.2125	0.2125	0.2125	0.2125
Elasticnet	정확도	0.489	0.468	0.421	0.094	0.0	0.0	0.0
	RMSE	0.1518	0.1550	0.1616	0.2022	0.2125	0.2125	0.2125

2.2 랜덤포레스트

랜덤 포레스트(Random Forest)는 주어진 학습 데이터에 따라 생성되는 의사결정트리가 매우 달라져 성능과 변동의 폭이 크다는 단점을 보완하기 위해서 만들어진 기법이다. 주어진 트레이닝 데이터 세트에서 무작위로 중복을 허용해 n 개를 선택하고 n 개의 데이터 샘플에서 데이터 특성값을 중복 허용 없이 d 개 선택하게 되고 이를 이용해 의사결정트리를 학습하고 생성한다. 이와 같은 방법을 k 번 반복하여 생성된 의사결정 트리를 이용해 예측하고, 예측된 결과의 평균이나 가장 많이 등장한 예측 결과를 최종 예측값으로 결정하는 알고리즘이다.

[그림 3-3] 랜덤포레스트의 알고리즘 구조



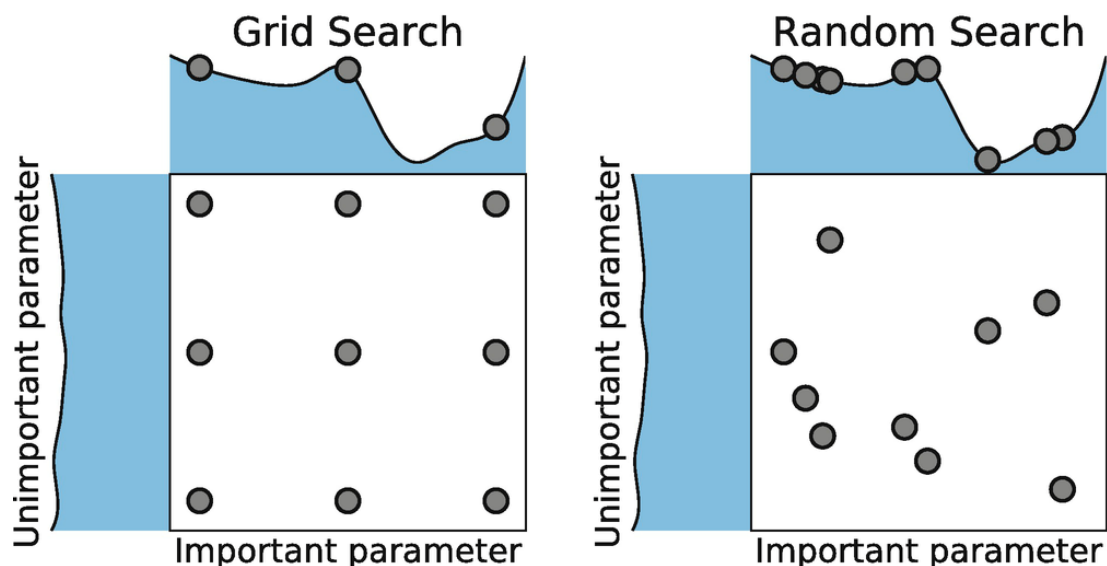
랜덤포레스트는 앙상블 기법으로 월등히 높은 정확성과 간편하고 빠른 학습 및 테스트 알고리즘, 변수소거 없이 수천 개의 입력 변수들을 다루는 것이 가능하다는 점 등 다양한 장점이 있어 이번 프로젝트에서 사용하게 되었다.

2.2.1 최적의 하이퍼파라미터

랜덤포레스트를 적용시킬 땐, 대표적인 하이퍼 파라미터인 'n_estimators', 'max_depth', 'min_samples_leaf', 'min_samples_split'를 사용하였다. 하이퍼 파라미터는 무작위성 제어를 위한 매개변수로, n_estimators는 트리의 개수를, max_depth는 복잡도 제어를 위한 매개변수로는 트리의 최대 깊이, min_samples_leaf 는 리프 노드가 되기 위한 최소한의 샘플 개수, min_samples_split 는 노드가 분기할 수 있는 최소 샘플 개수를 의미한다.

랜덤포레스트에서는 최적의 하이퍼파라미터를 선정하기 위해 GridSearchCV 알고리즘과 RandomSearchCV 알고리즘을 사용하였는데, GridSearchCV 알고리즘은 지정한 영역 내에서 모든 조합에 대해 교차검증 후 가장 좋은 성능을 내는 하이퍼파라미터 조합을 찾아내는 알고리즘이며 RandomSearchCV는 GridSearch 와 동일한 방식으로 사용하지만 모든 조합을 다 시도하지는 않고, 각 반복마다 임의의 값만 대입해 지정한 횟수만큼 평가하는 알고리즘이다.

[그림 3-4] GridSearchCV와 RandomSearchCV



GridSeachCV를 사용할때, ParamGrid를 'n_estimators': [100, 500, 1000, 1500], 'max_depth': [5, 10, 15, 20], 'min_samples_leaf': [8, 16, 20], 'min_samples_split': [8, 16, 20]를 사용하였으며 RandomSearchCV 'n_estimators' : 1~10000, 'max_depth' : 5~40, 'min_samples_leaf' : 8~20, 'min_samples_split' : 8~20사이의 범위로 설정하였으며 cv = 50으로 설정하여 최적의 하이퍼파라미터를 찾도록 하였다. 두가지의 알고리즘을 통해 'n_estimators'는 클 수록 좋았음을 알 수 있었다.

2.2.2 모델 성능 비교

GridSearchCV를 통해 하이퍼파라미터가 'max_depth'가 10일때, 'min_samples_leaf'가 8일때, 'min_samples_split'가 20일 때, 'n_estimators'가 1000일 때 최적이었음을 알 수 있었다. 설정한 param Grid의 경우의 수만큼 돌려보는 GridSearchCV와는 달리 RandomSearchCV는 범위 내에서 임의의 값을 찾게 되고 그 중 가장 좋은 성능을 보였다. 따라서 확실히 GridSearchCV 보다 실행 속도가 빨랐지만, 값이 매번 다르게 나와 여러 번 실행해 보았다.

여러 번 실행시켜 보니, 'max_depth', 'min_samples_leaf', 'min_samples_split'의 경우 10~20 사이의 값이 중복적으로 나왔고, 'n_estimators'의 경우 다양하게 나왔다. 하지만, 하이퍼파라미터에 따라 성능 부분에서 크게 개선되지 않았다. 가장 좋은 예측성능을 보였을 때에는 'max_depth'는 11, 'min_samples_leaf'는 14, 'min_samples_split'는 15, 'n_estimators'는 9083로 설정하였을 때였으며, 각 알고리즘 별로 선정된 최적의 하이퍼파라미터와 모델 정확도와 RMSE는 아래의 [표 3-3]과 같다.

[표 3-3]. GridSearchCV와 RandomSearchCV 별 하이퍼파라미터 및 모델 성능과 RMSE

	하이퍼파라미터	모델 성능 (정확도)		RMSE
GridSearchCV	'max_depth': 10, 'min_samples_leaf': 8, 'min_samples_split': 20, 'n_estimators': 1000	훈련데이터	0.62979	0.129040
		검증데이터	0.61966	0.131075

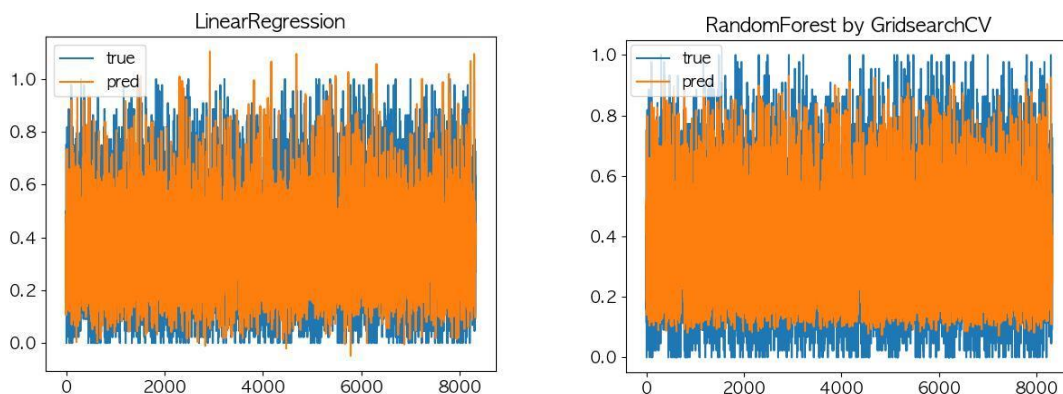
RandomSearchCV	'max_depth': 11, 'min_samples_leaf': 14, 'min_samples_split': 15, 'n_estimators': 9083	훈련데이터	0.630976	0.128834
		검증데이터	0.618280	0.131314

제 4장 결론

제 1절 연구 결과 종합 및 시사점 도출

이번 프로젝트를 통해, 정규화 및 표준화를 거친 데이터로 배달 소요시간에 가장 많이 영향을 미치는 변수가 어떤 것들인지 알 수 있었으며 과적합을 방지하기 위해 단계적 선택방법과 랜덤포레스트의 중요 변수 선출 기법과 같은 합리적인 방식을 통해 선정하였음에 의의가 있다. 전체적으로 오차가 적고 정확한 예측에 성공하지는 못하였지만 49%의 정확도를 나타냈던 선형회귀 방식과는 비교하여 앙상블 기법이었던 랜덤포레스트를 통해 정확도를 60%로 끌어올릴 수 있었다. 또한 성능을 더 높이기 위해 최적의 하이퍼파라미터를 찾기 위해 GridSearchCV와 RandomSearchCV를 동시에 사용하였으며, 최종적으로 검증데이터셋에서 0.133048로 가장 낮은 RMSE를 기록한 GridSearchCV 하이퍼파라미터를 적용시킨 랜덤포레스트가 최고의 모델이었다.

[그림 4-1] LinearRegression과 GridSearchCV - RandomForest의 모델 성능 시각화



제 2절 한계점 및 향후 연구 방향성 제언

2.1 한계점

본 연구는 다음과 같은 한계를 지닌다. 첫째, 다양한 변수 선출 방식을 선택하지 못하였다. 본 연구에서 사용한 방법을 제외하고 보루타 알고리즘과 같이 다양한 변수 선출 방식이 있는데, 두 가지 방법으로만 변수를 선출하였다는 한계를 지닌다. 랜덤 포레스트에서 도출한 변수들 중 중요도는 낮게 평가됐지만 실제로는 중요한 특성이 있는 변수가 있을 수 있다. 변수 중요도는 절대적인 수치가 아니기 때문에 중요도가 낮은 것을 다 제거한 뒤 중요도만을 기준으로 변수를 선정했을 때 모델 예측력이 낮게 나올 수 있다. 둘째, 배달 주문이 몰리는 시간(피크타임)을 변수로 새로 생성하여 고려하지 못하였다. 피크타임 변수의 경우 교호작용을 기대할 수 있는데, 교호작용이란 한 요인의 효과가 다른 요인의 수준에 의존하는 경우를 말한다. 예를 들어 피크타임 변수의 경우 주말 변수와 저녁시간대의 변수를 결합함으로써 피크타임 변수에 중요한 영향을 끼칠 수 있다. 배달 주문은 주문이 많이 몰리는 시간대의 영향력이 크기 때문에 배달 시간을 예측할 때 해당 변수를 고려하지 못하였다는 한계를 지닌다.

2.2 향후 연구 방향성 제언

위와 같은 한계점들을 보완하기 위해 다음과 같은 향후 연구를 진행해볼 수 있다. 보루타 알고리즘을 활용하여 변수 선출 방식을 달리한 뒤 모델의 성능을 평가해볼 수 있다. 보루타 알고리즘이란, 랜덤포레스트를 기반으로 변수를 선택하는 래퍼 방법(Wrapper Method)으로 기본적인 아이디어는 기존 변수를 복원추출해서 만든 변수(shadow) 보다 모형 생성에 영향을 주지 못했다고 하면 이는 가치가 크지 않은 변수로 인식하여 제거하는 방식을 말한다. 또한, 피크타임 변수와 같이 배달 소요 시간에 많은 영향을 끼치는 다른 변수들을 새로 생성하여 배달 시간을 예측해볼 수 있다. '대학내일20대연구소'에서 진행한 <19~34세 식생활 및 식문화 연구 보고서>에 따르면, 배달음식을 주로 먹는 시간대는 '저녁 6~9시'가 55.8%로 1위였고, 그 다음은 '저녁 9~12시(28.8%)'로 나타났다. 배달음식을 먹는 요일은 '토요일(68.4%)', '금요일(56.6%)', '일요일(42.3%)'순으로 집중되어 있었다.¹ 이러한 측면에서 볼 때 배달 시간에 피크타임 변수가 미치는 영향을 무시할 수 없을 것이기에 향후 연구에서는 피크타임 변수를 고려해볼 필요가 있을 것이다.

¹ <https://www.20slab.org/Archives/27030>

참고문헌

Kwon, J. et al. (2015) “A Study on the Number of Domestic Food Delivery Services,” Korean Journal of Applied Statistics. The Korean Statistical Society. doi: 10.5351/kjas.2015.28.5.977.

Yi, C.-D. (2021, March 31). Machine Learning Prediction of Economic Effects of Busan’s Strategic Industry through Ridge Regression and Lasso Regression. Journal of Korea Port Economic Association. The Korea Port Economic Association. <https://doi.org/10.38121/kpea.2021.03.37.1.197>

Shin, J. (2021). Assessment of Classification Accuracy of fNIRS-Based Brain-computer Interface Dataset Employing Elastic Net-Based Feature Selection. Journal of Biomedical Engineering Research, 42(6), 268–276. <https://doi.org/10.9718/JBER.2021.42.6.268>